

DSA STUDY BLUEPRINT SERIES

35. Recursion Basics to Advanced Part1

Introduction to Recursion Base Cases and Call Stacks

Section 05 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Recursion:** Functions calling themselves recursively.
- **Base Case:** Essential stopping rules to prevent stack overflows.
- **Stack Frames:** Memory frames allocated for functions.

Memory & Execution Blueprint



Call stack frames execution order (Top-down)

C++ Implementation Reference

Standard code layout and syntax

```
void printN(int n) {  
    if (n == 0) return; // Base Case  
    cout << n << " ";  
    printN(n - 1); // Recursive Call  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(N)$ (Recursion Stack)

Key Practices & Gotchas

- **Important:** Verify base cases first before coding recursive paths.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run recursion call stack execution for factorial function `fact(3)` :

Stack Level	Active Recursive Function Call	Base Condition Check	Value Resolution
1	<code>fact(3) = 3 * fact(2)</code>	False	Awaiting <code>fact(2)</code> return
2	<code>fact(2) = 2 * fact(1)</code>	False	Awaiting <code>fact(1)</code> return
3	<code>fact(1) = 1</code>	True (Base case reached!)	Returns 1

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Tail Recursion:** Compiler optimizations discard caller stack frames when recursive call is the final statement.
- **Call Stack limits:** Every recursive call consumes stack frames; excessive levels trigger stack overflows.
- **Related LeetCode Problems:** #509 Fibonacci Number, #344 Recursive String Reversals.